FILE HANDLING :

28 February 2026 10:15

⇒ Module 8.1 – File Class in Java

"Why do we need the File class when we can manually create files?"

1. Manual creation is not scalable or practical

Imagine you're writing a Java program that:

- Logs user activity into a daily file
- Loads configuration from a .properties file
- Checks if a report file already exists
- Deletes temp files after use

  You **can't do that manually** every time. You need **Java code to handle it**.

2. Automation & Real-world Examples

Scenario	Manual Possible?	Java File Needed?
Create daily backup files programmatically	No	Yes
Check if user uploaded file exists	No	Yes
Delete a temp log file after processing	No	Yes
Get file size to decide if compression needed	No	Yes
List all files in a directory	No	Yes

3. Java programs run on servers/cloud – not your PC

When your code runs:

- On a **web server**, **cloud platform**, or **CI/CD pipeline**
- There's **no human to manually create folders or files**
- Your code has to **create, manage, check, or delete** files itself

So, we use:

```
File file = new File("logs/2025-07-07.log");
if (!file.exists()) {
        file.createNewFile(); // Automatically create file for today
}
```

Final Thought:

- "Manually creating a file is fine **for you as a human**. But Java code must behave **like a smart robot** — it has to **check, create, and manage files on its own**. That's where File class shines!"

Real-Life Analogy

- **Think of File class like a file manager or librarian.**
- You don't open each book to check if it exists, right? You just look at its label, location, and size.

Similarly, Java's File class doesn't read the content — it lets you **check, create, delete, rename**, or get info about **files**

or folders.

✧ What is File Class?

- File is a class in the java.io package.
- It represents a file or directory (folder).
- It does **not** read/write data. Instead, it:
  - Checks if file/folder exists
  - Creates file/folder
  - Deletes them
  - Gets information like name, size, path, etc.

Basic Syntax

```
import java.io.File;

public class FileInfo {
    public static void main(String[] args) {
        File f = new File("test.txt");

        if (f.exists()) {
            System.out.println("Name: " + f.getName());
            System.out.println("Path: " + f.getAbsolutePath());
            System.out.println("Size: " + f.length() + " bytes");
            System.out.println("Readable: " + f.canRead());
            System.out.println("Writable: " + f.canWrite());
        } else {
            System.out.println("File does not exist.");
        }
    }
}
```

❖ Creating a New File

```
import java.io.File;
import java.io.IOException;

public class CreateFile {
    public static void main(String[] args) {
        try {
            File file = new File("demo.txt");
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

```
// Create a file inside  folder :

    File file = new File(folder, "myfile.txt");
    if (file.createNewFile()) {
```

```
        System.out.println("File created: " + file.getAbsolutePath());
    } else {
        System.out.println("File already exists.");
    }
```

❖ Commonly Used File Class Methods

Method	Description
◆ exists()	◆ Checks if file/folder exists
◆ createNewFile()	◆ Creates a new file
◆ mkdir()	◆ Creates a single directory
◆ mkdirs()	◆ Creates nested directories
◆ delete()	◆ Deletes file or empty folder
◆ getName()	◆ Returns name of file/folder
◆ getAbsolutePath()	◆ Full path of file/folder
◆ length()	◆ Size in bytes
◆ canRead() / canWrite()	◆ Check permissions
◆ isFile() / isDirectory()	◆ Is it a file or a folder?
◆ list()	◆ Lists all files/folders inside a directory

❖ Program all in one:

```
import java.io.File;
import java.io.IOException;

public class FileClassDemo {
    public static void main(String[] args) {
        try {
            // 1. Create a file object pointing to a file
            File file = new File("example.txt");

            // 2. Check if the file exists
            System.out.println("File exists? " + file.exists());

            // 3. Create the file if it does not exist
            if (!file.exists()) {
                boolean created = file.createNewFile(); // createNewFile()
                System.out.println("File created? " + created);
            }

            // 4. Get file name
            System.out.println("File Name: " + file.getName()); // getName()

            // 5. Get absolute path
            System.out.println("Absolute Path: " + file.getAbsolutePath()); // getAbsolutePath()

            // 6. Check read/write permissions
            System.out.println("Can Read? " + file.canRead());   // canRead()
            System.out.println("Can Write? " + file.canWrite()); // canWrite()

            // 7. Check size of file in bytes
            System.out.println("File Size: " + file.length() + " bytes"); // length()

            // 8. Check if it's a file or directory
```

```java
        System.out.println("Is File? " + file.isFile());        // isFile()
        System.out.println("Is Directory? " + file.isDirectory());// isDirectory()

        // 9. Create a single directory
        File folder = new File("MyFolder");
        if (!folder.exists()) {
            boolean folderCreated = folder.mkdir(); // mkdir()
            System.out.println("Single folder created? " + folderCreated);
        }

        // 10. Create nested directories
        File nestedFolder = new File("ParentFolder/ChildFolder/SubFolder");
        if (!nestedFolder.exists()) {
            boolean nestedCreated = nestedFolder.mkdirs(); // mkdirs()
            System.out.println("Nested folders created? " + nestedCreated);
        }

        // 11. List all files/folders inside ParentFolder
        File parent = new File("ParentFolder");
        if (parent.exists() && parent.isDirectory()) {
            String[] fileList = parent.list(); // list()
            System.out.println("Files inside ParentFolder:");
            for (String name : fileList) {
                System.out.println(" - " + name);
            }
        }

        // 12. Delete a file
        // Note: File must exist and not be open in another program
        boolean fileDeleted = file.delete(); // delete()
        System.out.println("File deleted? " + fileDeleted);

        // 13. Delete empty folder
        boolean folderDeleted = folder.delete(); // delete()
        System.out.println("Folder deleted? " + folderDeleted);

    } catch (IOException e) {
        System.out.println("An error occurred: " + e.getMessage());
    }
  }
}
```

⇒ Module 8.2 – FileReader & FileWriter in Java

Real-Life Analogy

Imagine writing a diary (FileWriter) and later **reading it back (FileReader)**.

- You use a **pen to write** (FileWriter).
- You use your **eyes to read** (FileReader).

Simple, right?

Same way in Java:

- FileWriter → writes text to a file
- FileReader → reads text from a file (character by character)

✧ **What is FileWriter?**

- Used to **write characters** into a text file.
- Comes from java.io.FileWriter
- Overwrites the file by default (can be set to append)

✧ **What is FileReader?**

- Used to **read characters** from a text file.
- Comes from java.io.FileReader
- Reads **character by character**, not line by line (that's BufferedReader)

✧ **Syntax: FileWriter**

```
FileWriter fw = new FileWriter("file.txt");
fw.write("Hello students!");
fw.close();
```

✧ **Syntax: FileReader**

```
import java.io.FileReader;

FileReader fr = new FileReader("file.txt");
int ch;
while ((ch = fr.read()) != -1) {
        System.out.print((char) ch);
        }
        fr.close();
```

❖ Full Example: Writing and Reading a File

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class FileReadWriteDemo {
    public static void main(String[] args) {
        try {
            // Writing to a file
            FileWriter writer = new FileWriter("students.txt");
            writer.write("Hello Java Students!\nWelcome to File Handling.");
            writer.close();
            System.out.println(" Data written successfully.");

            // Reading from the file
            FileReader reader = new FileReader("students.txt");
            System.out.println(" File content:");
            int ch;
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println(" Error occurred: " + e.getMessage());
        }
    }
```

```
}
```

❖ Common Methods of FileWriter & FileReader

Class	Method	Description
◆ FileWriter	◆ write(String)	◆ Writes string to file
	◆ write(char)	◆ Writes a single character
	◆ write(char[])	◆ Writes array of characters
	◆ close()	◆ Closes the writer
◆ FileReader	◆ read()	◆ Reads one character (returns int)
	◆ read(char[])	◆ Reads into a char array
	◆ close()	◆ Closes the reader

❖ Summary

- FileWriter is for writing **character-based data** to a file.
- FileReader is for reading **character-by-character** from a file.
- Always close files to avoid data corruption or memory leaks.

⇒ Module 8.3 – BufferedReader & BufferedWriter in Java

Real-Life Analogy

- Imagine you're **reading a novel** 📖
- You don't read **letter by letter** — you read **line by line**, right?

Likewise in Java:

- FileReader reads **character by character** (slow).
- BufferedReader reads **line by line**, more efficiently
  Same for writing — instead of writing every character instantly (like a slow typist), BufferedWriter writes in **chunks**, making it faster.

❖ What is BufferedReader?

- Reads **text from a character input stream**, efficiently.
- Wraps around FileReader for **fast line-by-line reading**.
- Has method: readLine()

❖ What is BufferedWriter?

- Writes text to a character output stream efficiently.
- Wraps around FileWriter.
- Has method: write(), newLine()

◆ Syntax: BufferedReader

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));
String line;
while ((line = br.readLine()) != null) {
        System.out.println(line);
}
        br.close();
```

- **Syntax: BufferedWriter**

```
import java.io.BufferedWriter;
import java.io.FileWriter;

BufferedWriter bw = new BufferedWriter(new FileWriter("data.txt"));
bw.write("Welcome students!");
bw.newLine(); // writes a line break
bw.write("This is buffered writing.");
bw.close();
```

❖ **Full Example: BufferedReader & BufferedWriter**

```
import java.io.*;

public class BufferedReadWriteDemo {
    public static void main(String[] args) {
        try {
            // Writing to file using BufferedWriter
            BufferedWriter bw = new BufferedWriter(new FileWriter("info.txt"));
            bw.write("Java is fun.");
            bw.newLine();
            bw.write("BufferedReader and BufferedWriter are cool!");
            bw.close();
            System.out.println("Writing done using BufferedWriter.");

            // Reading from file using BufferedReader
            BufferedReader br = new BufferedReader(new FileReader("info.txt"));
            String line;
            System.out.println(" Reading file content:");
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
            br.close();

        } catch (IOException e) {
            System.out.println(" Error: " + e.getMessage());
        }
    }
}
```

❖ Copy from one file to **another** file :

```
Path source = Paths.get("E://jatin.txt");
Path destination = Paths.get("E://java.txt");

try {
```

```
    Files.copy(source, destination, StandardCopyOption.REPLACE_EXISTING);

    System.out.println("File content copied successfully using Files.copy()");
} catch (IOException e) {
    e.printStackTrace();
}
```

❖ **Example 1: Renaming a File**

```
Path source = Paths.get("E://oldname.txt");
Path target = Paths.get("E://newname.txt");

try {
    Files.move(source, target);
    System.out.println(" File renamed successfully.");
} catch (Exception e) {
    e.printStackTrace();
}
```

❖ **Example 2: Moving File to Another Folder**

```
Path source = Paths.get("E://file.txt");
Path target = Paths.get("E://backup//file.txt");

try {
    Files.move(source, target, StandardCopyOption.REPLACE_EXISTING);
    System.out.println("📄 File moved to backup folder.");
} catch (Exception e) {
    e.printStackTrace();
}
```

✧ **<u>Common Methods of BufferedReader & BufferedWriter</u>**

Class	Method	Description
◆ BufferedReader	◆ readLine()	◆ Reads one full line at a time
	◆ close()	◆ Closes the stream
◆ BufferedWriter	◆ write(String)	◆ Writes string to file
	◆ newLine()	◆ Inserts a platform-independent line break
	◆ flush()	◆ Flushes the stream (forces write)
	◆ close()	◆ Closes the stream

✧ **<u>Why Use BufferedReader/Writer Instead of FileReader/Writer?</u>**

Feature	FileReader/FileWriter	BufferedReader/BufferedWriter
◆ Speed	◆ Slower	◆ Faster (uses internal buffer)
◆ Read type	◆ Character by character	◆ Line by line
◆ Control over line breaks	◆ No	◆ Yes (with newLine())
◆ Suitable for	◆ Small tasks	◆ Large files / performance

⇒ <u>Module 8.4 – Scanner for File Input in Java</u>

Real-Life Analogy

- Imagine you're scanning a document **line-by-line, word-by-word, or number-by-number** using a digital scanner.
- That's exactly what the **Scanner class** does when used with a file!

- BufferedReader reads line-by-line.
- But Scanner goes one step further:

*It allows you to **read individual tokens** — **words, numbers, lines**, etc.*

❖ What is Scanner (for File)

- Scanner is part of java.util package.
- Normally used to read user input (like Scanner sc = new Scanner(System.in)).
- But it can also read **from a file** — **line-by-line, word-by-word, int-by-int**, etc.

❖ How to Create Scanner for File

```
Scanner sc = new Scanner(new File("data.txt"));
```

❖ Methods of Scanner (for File)

Method	Description
next()	Reads next word/token (until space)
nextLine()	Reads the entire line
nextInt()	Reads next integer
nextDouble()	Reads next double
hasNext()	Checks if there is another word/token
hasNextLine()	Checks if there is another line
hasNextInt()	Checks if next token is integer

Full Example: Reading a File using Scanner

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ScannerFileExample {
    public static void main(String[] args) {
        try {
            // Creating Scanner to read file
            File file = new File("scannerdata.txt");
            Scanner sc = new Scanner(file);

            System.out.println(" File Content (word-by-word):");
            while (sc.hasNext()) {
                String word = sc.next(); // Reads word-by-word
                System.out.println(word);
            }

            sc.close();

        } catch (FileNotFoundException e) {
```

```
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
```

❖ **Difference Table: FileReader vs BufferedReader vs Scanner (for File Input)**

Feature / Use Case	FileReader	BufferedReader	Scanner
Purpose	Read file character by character	Read file line by line efficiently	Read file word, number, or line
Reads	Character by character	Line by line (readLine())	Token by token, line, word, int, etc.
Speed	Slow	Fast (has internal buffer)	Slower than BufferedReader
Reads numbers	Not supported	Not supported	Supported (nextInt(), nextDouble())
Reads full lines	Not directly (only read())	Yes (readLine())	Yes (nextLine())
Best for	Simple character reading	Big text files, log files	Simple parsing, line or token reading
Reads by tokens	Not supported	Not supported	Supported
Customization	Low	Medium	High (regex, delimiter)
Close required	Yes (close())	Yes (close())	Yes (close())
Example use case	Show file content character-wise	Read log files line-by-line	Read student data with name, marks